UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL MAR DEL PLATA

Tecnicatura Universitaria en Programación

Asignatura: Metodología de Sistemas II — 2026

Trabajo Práctico Final

Auditoría de código, patrones de diseño y modelado UML

Proyecto auditado: AlojaME — Plataforma de reserva de alojamientos

Integrante: Eloy Roldán

Docente: Rómulo Currao

Ciclo lectivo: 2026

Entrega de documentación: 17/11/2026 — Exposición: 18/11/2026

Versión del documento: v0.1

Índice

1. Breve descripción del dominio

2. Composición del equipo

3. Detalle de arquitectura

4. Estado actual del código y propuestas de mejora

5. Diagramas UML y pruebas

6. Conclusiones

Referencias

1. Breve descripción del dominio

AlojaME es una plataforma web de publicación y reserva de alojamientos temporarios, con un modelo tipo marketplace: los anfitriones publican sus propiedades, los clientes las reservan y pagan a través de la plataforma, y la plataforma retiene el dinero y lo liquida al anfitrión una vez cumplida la estadía. El sistema integra pagos reales mediante MercadoPago, verificación de cuentas por correo electrónico (Brevo) y almacenamiento de imágenes en la nube (Cloudinary).

Requerimientos generales:

- **Cuentas y seguridad:** registro con verificación de email mediante token de un solo uso (24 h), inicio de sesión con JWT y tres roles (Cliente, Anfitrión y Administrador).
- **Catálogo:** publicación de alojamientos con dirección, tipo, servicios y galería de imágenes; navegación y búsqueda públicas sin necesidad de cuenta.
- **Reservas:** solicitud con control de superposición de fechas; ciclo de vida SOLICITADA → CONFIRMADA → FINALIZADA, con cancelación posible desde SOLICITADA o CONFIRMADA.
- **Pagos:** al confirmar se genera el pago con un plazo de 48 h; el cliente abona el monto del anfitrión más la comisión de la plataforma (5 %) y la comisión estimada de MercadoPago; al finalizar, la plataforma liquida al anfitrión (payout).
- **Cancelaciones:** si cancela el anfitrión, reembolso total; si cancela el cliente con 48 h o más de anticipación, reembolso total; con menos de 48 h, reembolso del 65 % y el 35 % restante para el anfitrión; el administrador puede definir un reembolso a medida.
- **Reseñas y estadísticas:** reseña por reserva finalizada y tableros de resumen por rol (gastos del cliente, ingresos del anfitrión, flujo de dinero global).

2. Composición del equipo

Tabla 1. Composición del equipo.

Integrante	**Participación en esta auditoría**	**¿Participó en la autoría del proyecto?**
Eloy Roldán	Análisis del código, propuestas de mejora, diagramas UML, diseño de pruebas y redacción	Sí. Autor del backend (Programación III) y del frontend Angular (Programación IV)

El trabajo se realiza de forma individual. Como el auditor es también el autor del sistema, se adoptó un criterio explícito para reducir el sesgo: cada propuesta se respalda con evidencia verificable en el código (clase, método y línea) y con métricas objetivas de tamaño y acoplamiento, en lugar de apreciaciones generales.

3. Detalle de arquitectura

3.1 Lenguaje y frameworks

Tabla 2. Lenguajes, frameworks y servicios utilizados.

Capa	Tecnología
Backend	Java 21 · Spring Boot 3.4.3 (Web, Data JPA, Security, Validation) · Hibernate · JJWT 0.12.6 · Lombok · Springdoc OpenAPI (Swagger) · Spring Actuator + Micrometer
Persistencia	MariaDB / MySQL (esquema testdb, 12 tablas), mapeado con JPA
Frontend	Angular 22 (componentes standalone, TypeScript), CSS propio
Servicios externos	MercadoPago (Checkout Pro, reembolsos, payouts) · Brevo (email transaccional por API HTTP) · Cloudinary (almacenamiento de imágenes)
Herramientas	Git y GitHub · Maven · Angular CLI · Jira (Scrum)

3.2 Repositorios

El código se versiona en un repositorio privado de GitHub. El repositorio contiene el backend auditado, el frontend Angular y la documentación del proyecto. El backend se organiza en los paquetes Config, Converter, DTO, Enum, Exception, Model, Repository, RestController, Service y Validation.

3.3 Arquitectura web

El sistema sigue una arquitectura cliente-servidor. La SPA Angular consume una API REST sin estado (JSON sobre HTTPS, con el JWT en el encabezado Authorization). El backend está organizado en capas Controller → Service → Repository: los controladores solo traducen HTTP, los servicios concentran las reglas de negocio y las transacciones, y los repositorios encapsulan el acceso a datos. Nunca se exponen entidades JPA: toda respuesta se arma como DTO dentro de un método transaccional.

Reglas de diseño ya vigentes en el proyecto, relevantes para la auditoría:

- Un servicio solo accede a su propio repositorio; si necesita datos de otra entidad, llama al servicio dueño.
- Las políticas de cancelación ya aplican el patrón Strategy (interfaz PoliticaCancelacion con una implementación por motivo) y la pasarela de pago aplica el principio de inversión de dependencias (interfaz PasarelaDePagoService).
- Las transiciones de estado concurrentes sobre una misma reserva se protegen con bloqueo pesimista.

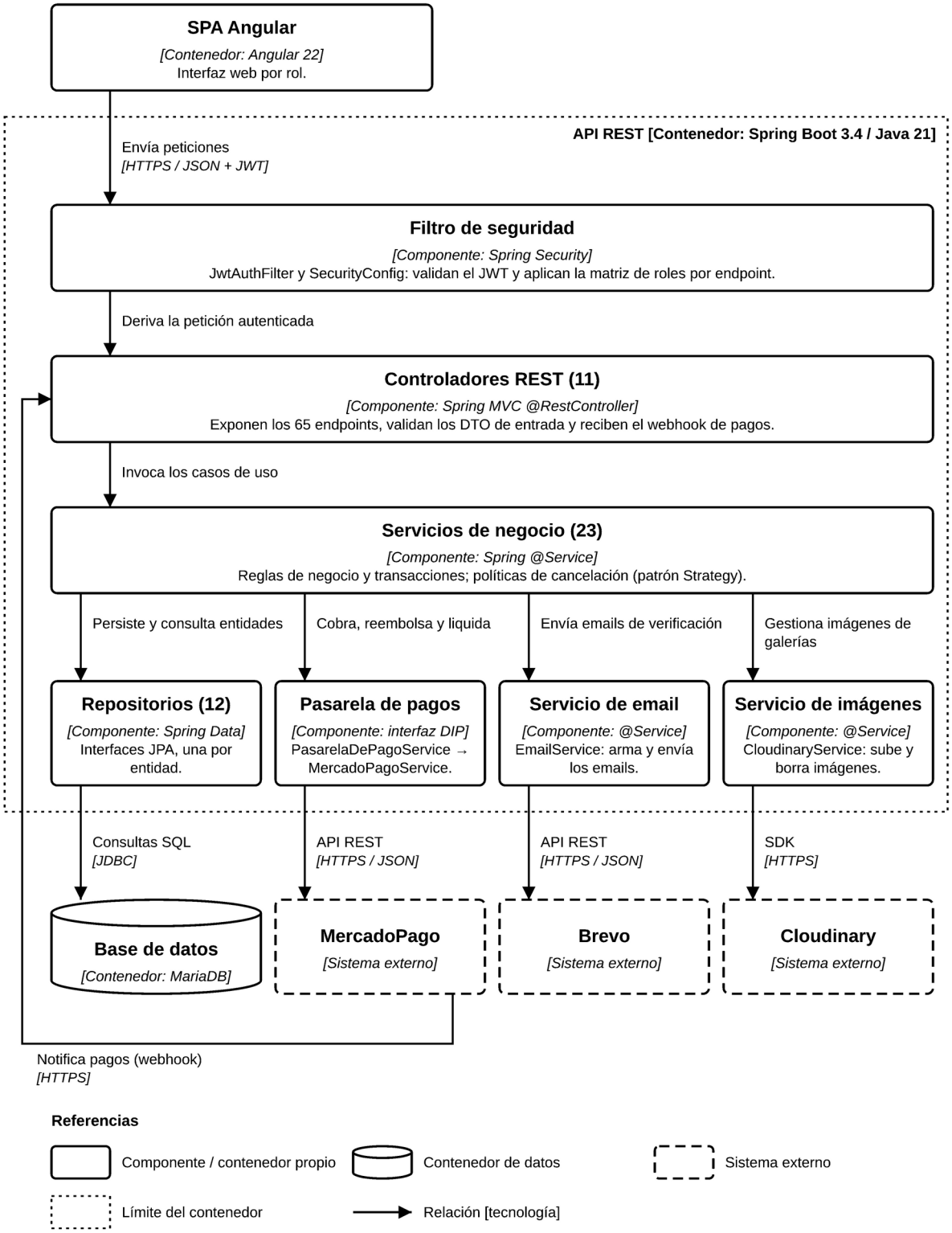


Figura 1. Componentes de la API REST y sus dependencias externas (notación C4).

4. Estado actual del código y propuestas de mejora

4.1 Método de relevamiento y métricas

Se revisaron los paquetes Service y RestController del backend. Para detectar clases saturadas y acoplamiento se midió el tamaño de cada clase (líneas de código), se relevaron las dependencias entre servicios (inyecciones y ciclos resueltos con @Lazy) y se buscaron reglas repetidas en varios lugares. Las mejoras que ya estaban aplicadas (Strategy en cancelaciones y DIP en la pasarela de pago) no se cuentan como propuestas: se usan como referencia del criterio esperado.

Tabla 3. Clases de mayor tamaño del backend.

Clase	Líneas	Observación
ReservaService	456	Transiciones de estado, validaciones, autorización y armado de DTO
PagoService	378	Pagos, webhook, cancelaciones, payouts, job programado y 7 consultas estadísticas
UsuarioService	247	Alta, verificación, perfil y alias de cobro
AlojamientoService	228	ABM de alojamientos y armado de DTO
MercadoPagoService	160	Adaptador de la API de pagos (ya detrás de una interfaz)
EstadisticasController	142	Depende de 5 servicios y contiene lógica de armado y de roles

Respecto de las grandes jerarquías de clases (punto ii de la plantilla), el dominio no presenta herencia relevante: las entidades JPA son planas y los roles se modelan con un enumerado. Por eso las propuestas se concentran en responsabilidades, acoplamiento, interfaces y patrones (puntos i, iii, iv y v). Los patrones se toman de Gamma et al. (1994), los principios de responsabilidad de Larman (2004) y los principios SOLID de Martin (2017).

4.2 Propuesta 1: patrón State para el ciclo de vida de la reserva

Tabla 4. Propuesta 1: patrón State para el ciclo de vida de la reserva.

Campo	Detalle
Módulo / paquete	Service — ReservaService (métodos actualizar, confirmar, cancelar, cancelarComoAdmin y finalizar).
Descripción del dominio	Una reserva pasa por SOLICITADA, CONFIRMADA, FINALIZADA o CANCELADA, y cada operación solo es válida en ciertos estados (ver el diagrama de estados, Figura 4).
Problema detectado	Cada operación repite su propio chequeo de estado con condicionales (líneas 159, 192, 215, 253 y 277): las reglas de transición están dispersas en cinco métodos de una clase de 456 líneas. Agregar un estado nuevo (por ejemplo, EN_CURSO durante la estadía) obliga a modificar todos esos métodos. Corresponde a los puntos i y v.
Solución a implementar	Patrón State: interfaz EstadoReserva con las operaciones confirmar, cancelar, finalizar y reprogramar, y una clase por estado. Una fábrica traduce el valor persistido (enum) a su clase. ReservaService conserva la transacción, el bloqueo pesimista y las llamadas a PagoService; cada estado solo decide si la transición es válida y cuál es el estado siguiente.

Campo	Detalle
Ventajas	Cumple el principio abierto/cerrado; las reglas de cada estado quedan en un solo lugar y se prueban de forma aislada; el diagrama de estados se refleja uno a uno en el código.
Posibles desventajas	Agrega cinco clases. El estado se sigue guardando como enum, por lo que la fábrica es imprescindible. Si se mueven también los efectos (pagos, reembolsos) a los estados, se mezcla responsabilidad y se arriesga duplicarlos: deben quedar en el servicio. Es viable porque es un cambio interno que no altera la API.

Esbozo de la solución:

```
interface EstadoReserva {
    ESTADORESERVA confirmar(Reserva r);   // devuelve el estado siguiente
    ESTADORESERVA cancelar(Reserva r);
    ESTADORESERVA finalizar(Reserva r);
}
class Solicitada implements EstadoReserva {
    public ESTADORESERVA confirmar(Reserva r) { return ESTADORESERVA.CONFIRMADA;
}
    public ESTADORESERVA finalizar(Reserva r) {
        throw new ParametroInvalidoException("Solo se finalizan reservas
CONFIRMADA."); }
    ...
}
```

4.3 Propuesta 2: eventos de dominio (Observer) entre Reserva y Pago

Tabla 5. Propuesta 2: eventos de dominio (Observer) entre Reserva y Pago.

Campo	Detalle
Módulo / paquete	Service — ReservaService y PagoService.
Descripción del dominio	Confirmar una reserva crea su pago; cancelarla dispara el reembolso según la política; finalizarla dispara la liquidación al anfitrión; y un pago vencido cancela la reserva.
Problema detectado	Existe una dependencia circular entre ambos servicios, resuelta con @Lazy en PagoService (línea 68). ReservaService invoca directamente siete métodos distintos de PagoService. Es un caso claro de alto acoplamiento entre módulos (punto iii).
Solución a implementar	Patrón Observer mediante eventos de aplicación de Spring: ReservaService publica ReservaConfirmada, ReservaCancelada y ReservaFinalizada, y PagoService los escucha con @TransactionalEventListener en fase BEFORE_COMMIT para conservar la atomicidad. En sentido inverso, el job de vencimiento publica PagoVencido y lo escucha ReservaService. Desaparece el @Lazy.
Ventajas	Elimina el ciclo de dependencias; nuevos interesados (por ejemplo, un email al cliente cuando se confirma su reserva) se agregan sin modificar ReservaService.
Posibles desventajas	El flujo se vuelve menos explícito y más difícil de depurar. La fase transaccional es crítica: si el listener corriera después del commit, un error al crear el pago dejaría una reserva CONFIRMADA sin pago. Requiere pruebas de integración que hoy el proyecto no tiene.

4.4 Propuesta 3: dividir responsabilidades de PagoService (SRP)

Tabla 6. Propuesta 3: dividir responsabilidades de PagoService (SRP).

Campo	Detalle
Módulo / paquete	Service — PagoService (378 líneas).
Descripción del dominio	La plataforma retiene el pago y lo liquida; el monto del cliente se calcula por gross-up a partir del monto que quiere recibir el anfitrión.
Problema detectado	La clase concentra al menos cinco responsabilidades: calcular montos y crear la preferencia, procesar el webhook, ejecutar cancelaciones y payouts, correr el job programado de vencimiento y resolver siete consultas estadísticas. Es una clase saturada de responsabilidades (punto i).
Solución a implementar	Extraer CalculadoraMontos (cálculo de comisiones y gross-up, sin estado), VencimientoPagosJob (solo la programación, delega en PagoService) y PagoConsultasService (consultas de solo lectura). Responde a los patrones GRASP de alta cohesión y fabricación pura (Larman, 2004).
Ventajas	Clases más pequeñas y cohesivas; el cálculo monetario, que es la parte más sensible, se prueba unitariamente sin dependencias.
Posibles desventajas	Choca con la regla actual del proyecto de que solo un servicio usa cada repositorio: habría que redefinirla por agregado (todo el paquete Pago). Agrega inyecciones y archivos. El riesgo funcional es bajo porque no cambia el contrato público.

4.5 Propuesta 4: interfaces y adaptadores para email e imágenes (DIP + Adapter)

Tabla 7. Propuesta 4: interfaces y adaptadores para email e imágenes (DIP + Adapter).

Campo	Detalle
Módulo / paquete	Service — EmailService, Galeria/CloudinaryService, Galeria/ImagenService y VerificacionService.
Descripción del dominio	La verificación de cuentas depende del envío de emails y las galerías dependen del almacenamiento de imágenes en la nube.
Problema detectado	EmailService está atado a la API de Brevo (URL fija en la línea 28) e ImagenService depende de la clase concreta CloudinaryService, que devuelve un Map sin tipo con claves propias del proveedor ("secure_url", "public_id"). El principio de inversión de dependencias ya se aplicó a los pagos, pero no a estos dos proveedores (punto iv).
Solución a implementar	Interfaces NotificadorEmail (enviar) y AlmacenamientoImagenes (subir y eliminar, devolviendo un record ImagenSubida con url e identificador externo). Las clases actuales pasan a ser adaptadores: BrevoEmailAdapter y CloudinaryAlmacenamientoAdapter.
Ventajas	Permite cambiar de proveedor o usar implementaciones falsas en pruebas sin tocar la lógica de negocio; el tipado reemplaza al Map genérico.
Posibles desventajas	Hoy existe una sola implementación de cada interfaz, por lo que puede verse como abstracción prematura. El costo del cambio es bajo y el impacto se limita a tres clases.

4.6 Propuesta 5: fachada para las estadísticas (Facade)

Tabla 8. Propuesta 5: fachada para las estadísticas (Facade).

Campo	Detalle
Módulo / paquete	RestController — EstadisticasController.

Campo	Detalle
Descripción del dominio	Cada rol tiene su tablero: el administrador ve flujo de dinero, rankings, cancelaciones e ingresos; el anfitrión, el resumen por alojamiento y los cobros pendientes; el cliente, sus gastos.
Problema detectado	El controlador depende de cinco servicios y contiene lógica de negocio: combina tres mapas para armar el resumen del anfitrión y decide por rol qué usuario consultar (líneas 92 a 140). Un controlador debería solo coordinar la petición (punto iii).
Solución a implementar	Patrón Facade: EstadisticasFacade, un servicio orquestador sin repositorio propio que expone un método por tablero y resuelve la regla de qué datos puede ver cada rol. El controlador queda reducido a traducir HTTP.
Ventajas	Controlador delgado; la lógica se prueba sin levantar la capa web; la fachada se puede reutilizar, por ejemplo para exportar reportes.
Posibles desventajas	Agrega una capa. Si concentra demasiadas operaciones puede convertirse en una clase dios: conviene dividirla por rol si crece.

4.7 Propuesta 6: cadena de validadores para crear y reprogramar reservas

Tabla 9. Propuesta 6: cadena de validadores para crear y reprogramar reservas.

Campo	Detalle
Módulo / paquete	Service — ReservaService, métodos crear y actualizar (líneas 87 a 175).
Descripción del dominio	Una reserva solo puede crearse si el alojamiento, el anfitrión y el cliente están activos, las fechas son coherentes y futuras y no se superponen con otra reserva.
Problema detectado	Las validaciones se encadenan con condicionales dentro de crear y parte de ellas se repite en actualizar. Hay reglas de producto pendientes (capacidad frente a cantidad de huéspedes, anticipación máxima) que harían crecer todavía más el método (punto v).
Solución a implementar	Patrón Chain of Responsibility: interfaz ValidadorReserva con una implementación por regla (alojamiento activo, fechas coherentes, fecha futura, sin superposición), inyectadas como lista y ordenadas con @Order. Es el mismo mecanismo que el proyecto ya usa para las políticas de cancelación.
Ventajas	Agregar una regla es agregar una clase; las mismas reglas se reutilizan al crear y al reprogramar; cada una tiene su prueba.
Posibles desventajas	La validación de superposición debe ejecutarse dentro de la transacción que ya tomó el bloqueo pesimista sobre el alojamiento; el orden de la cadena importa; para reglas triviales puede parecer excesivo.

Esbozo de la solución:

```
public interface ValidadorReserva { void validar(SolicitudReserva s); }

@Component @Order(10)
class FechasCoherentesValidador implements ValidadorReserva {
    public void validar(SolicitudReserva s) {
        if (!s.fechaFin().isAfter(s.fechaInicio()))
            throw new ParametroInvalidoException("La fecha de fin no puede
ser ...");
    }
}
// en ReservaService:  validadores.forEach(v -> v.validar(solicitud));
```

4.8 Propuesta 7: centralizar la autorización por recurso

Tabla 10. Propuesta 7: centralizar la autorización por recurso.

Campo	Detalle
Módulo / paquete	RestController (Alojamiento, Galeria, Review, Pago y Estadisticas) y Service (Reserva, Review y Alojamiento).
Descripción del dominio	Además de los tres roles, muchas operaciones exigen ser dueño del recurso: un cliente solo ve sus reservas y un anfitrión solo gestiona sus alojamientos.
Problema detectado	La comparación con TIPOUSUARIO.ADMIN y los chequeos de pertenencia aparecen en al menos nueve lugares entre controladores y servicios. Un cambio de regla, como un nuevo rol de soporte, obliga a recorrerlos todos (punto iii).
Solución a implementar	Concentrar las reglas de acceso en componentes de seguridad por recurso y aplicarlas con @PreAuthorize (habilitando la seguridad por método). Como primer paso, de bajo costo, agregar al propio Usuario un método esAdmin(), siguiendo el patrón experto en información.
Ventajas	Reglas de acceso en un único lugar, fáciles de auditar; menos repetición; el patrón variaciones protegidas aísla al resto del código de cambios en los roles.
Posibles desventajas	Las expresiones en anotaciones son menos legibles y se validan en tiempo de ejecución; hay que evitar consultar la base dos veces (una para autorizar y otra para operar); conviene migrar de forma gradual.

4.9 Resumen y priorización

Tabla 11. Resumen y priorización de las propuestas.

N.º	Propuesta	Punto	Impacto	Esfuerzo	Prioridad
1	State para el ciclo de la reserva	i, v	Alto	Medio	Alta
2	Eventos de dominio Reserva–Pago	iii	Alto	Medio	Alta
3	SRP en PagoService	i	Medio	Medio	Media
4	Adaptadores para email e imágenes	iv	Medio	Bajo	Media
5	Fachada de estadísticas	iii	Medio	Bajo	Media
6	Cadena de validadores de reserva	v	Medio	Bajo	Alta
7	Autorización centralizada	iii	Medio	Medio	Baja

5. Diagramas UML y pruebas

Los diagramas siguen la notación UML 2.5 (Object Management Group, 2017) y se acotan al subsistema central del dominio: reservas y pagos. Se elaboraron a partir del código real, de modo que cada elemento puede rastrearse hasta una clase o un método.

5.1 Diagrama de casos de uso

Muestra los tres actores humanos y los casos de uso del subsistema. Las relaciones «include» indican comportamiento que siempre forma parte del caso base (por ejemplo, reservar siempre verifica disponibilidad) y la relación «extend» indica el reembolso manual que el administrador agrega opcionalmente al cálculo del reembolso.

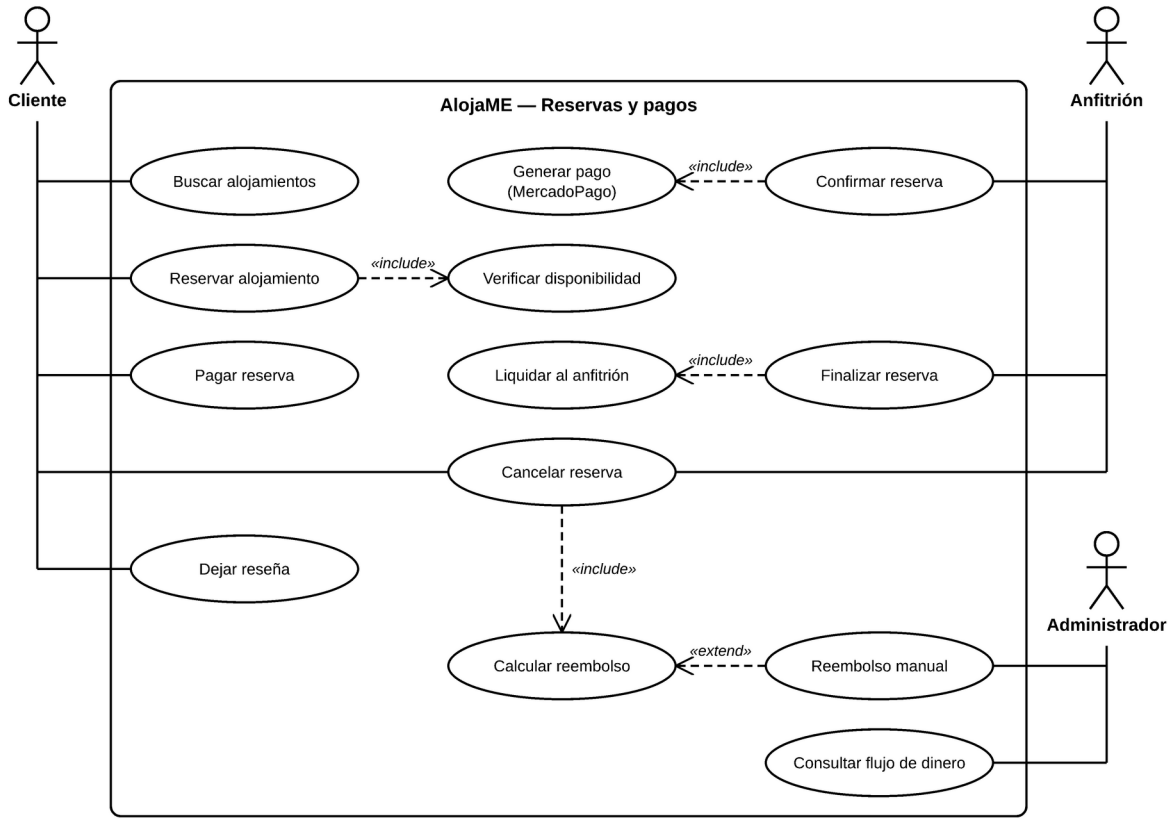


Figura 2. Diagrama de casos de uso del subsistema de reservas y pagos.

5.2 Diagrama de secuencia

Representa el escenario «confirmar una reserva»: la petición atraviesa el controlador y el servicio de reservas, que bloquea la fila, valida la operación y delega en el servicio de pagos

la creación de la preferencia en MercadoPago. Los mensajes de retorno se dibujan con línea discontinua.

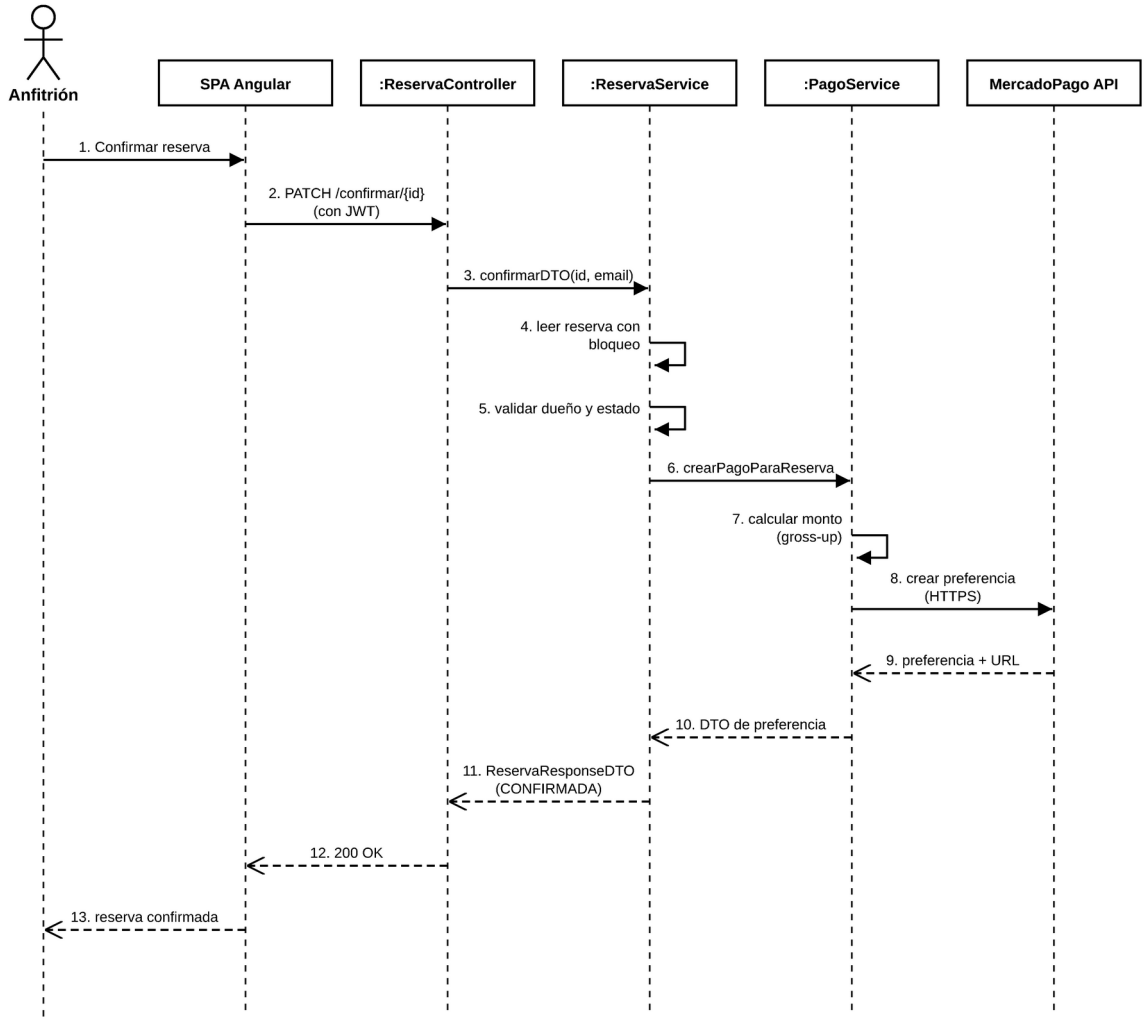
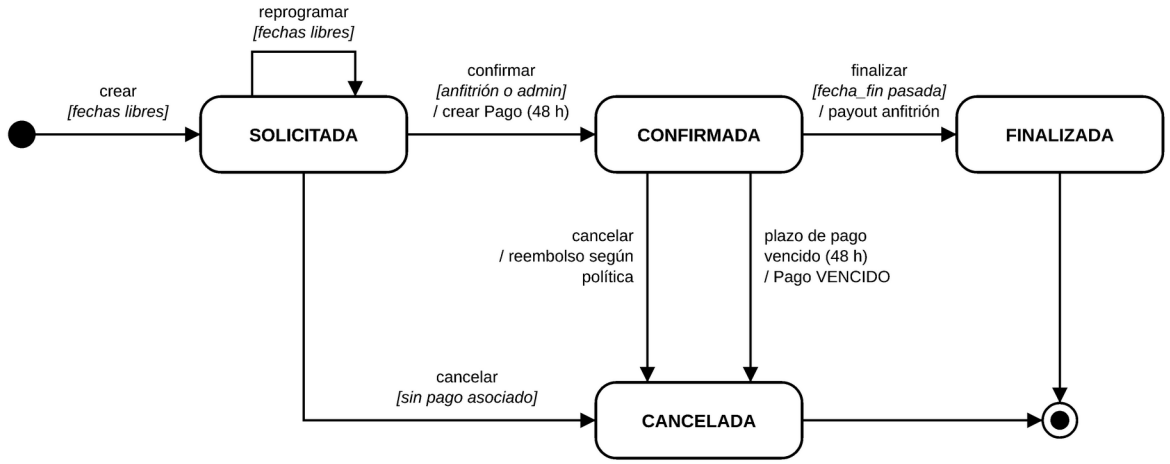


Figura 3. Diagrama de secuencia: confirmar una reserva y generar su pago.

5.3 Diagrama de estado

Modela el ciclo de vida de la entidad Reserva con la notación evento [condición] / acción. Es la base de la Propuesta 1 (patrón State).



Notación: evento [condición] / acción.

Figura 4. Diagrama de estados de la entidad Reserva.

5.4 Diagrama de actividades

Describe el proceso de negocio completo, desde la búsqueda hasta la liquidación al anfitrión, con un carril por participante y los puntos de decisión que bifurcan el flujo.

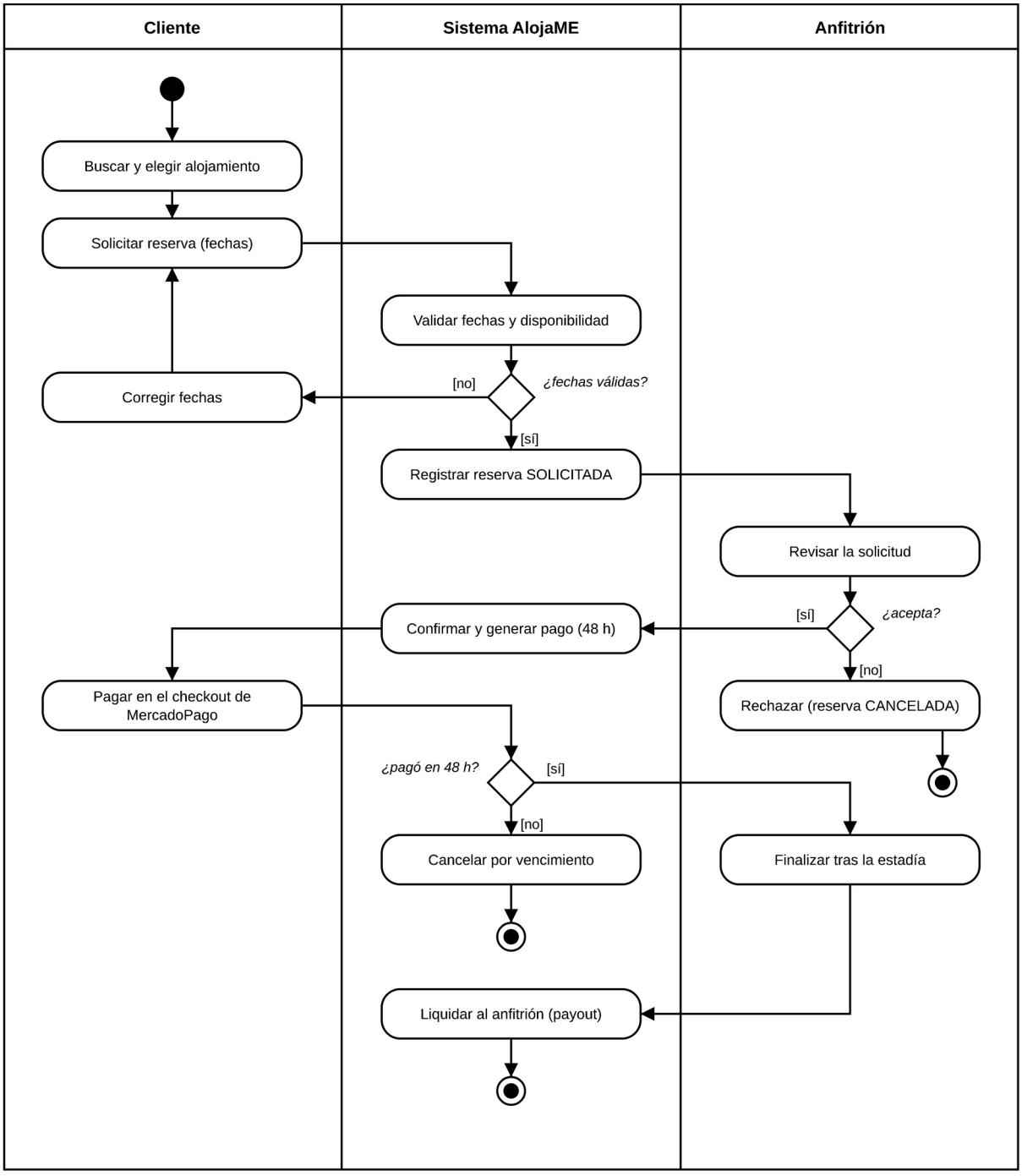


Figura 5. Diagrama de actividades: solicitar, confirmar y pagar una reserva.

5.5 Diseño de condiciones de prueba con técnicas de caja negra

Se aplican particiones de equivalencia y análisis de valores límite (Myers et al., 2011) sobre dos funcionalidades con reglas de negocio precisas: la creación de una reserva y la política de cancelación del cliente. Los resultados esperados se toman del comportamiento especificado y de los mensajes reales del sistema.

5.5.1 Crear reserva — particiones de equivalencia

Tabla 12. Clases de equivalencia para crear una reserva.

Variable	Clases válidas	Clases inválidas
fechaInicio	CV1: igual o posterior a hoy	CI1: anterior a hoy
fechaFin	CV2: posterior a fechaInicio	CI2: igual a fechaInicio · CI3: anterior a fechaInicio
Disponibilidad	CV3: sin reservas no canceladas que se superpongan	CI4: superposición con una reserva SOLICITADA o CONFIRMADA
Alojamiento	CV4: activo	CI5: inactivo

Tabla 13. Casos de prueba de caja negra para crear una reserva.

ID	Entrada	Clases	Resultado esperado
CP01	inicio = hoy + 10; fin = hoy + 13; libre; activo	CV1–CV4	Reserva creada en estado SOLICITADA
CP02	inicio = hoy − 1	CI1	Error: «La fecha de inicio no puede ser anterior al dia actual.»
CP03	fin = inicio	CI2	Error: «La fecha de fin no puede ser anterior o igual a la fecha de inicio.»
CP04	fin = inicio − 2	CI3	Mismo error que CP03
CP05	fechas superpuestas con una reserva CONFIRMADA	CI4	Error: «Ya existe una reserva en esas fechas»
CP06	fechas superpuestas solo con una reserva CANCELADA	CV3	Reserva creada (las canceladas no bloquean)
CP07	alojamiento inactivo	CI5	Error: «El alojamiento esta inactivo»
CP08	inicio = hoy; fin = hoy + 1 (valores límite)	CV1, CV2	Reserva creada por una noche

5.5.2 Cancelación por el cliente — valores límite

Regla: la cancelación del cliente es tardía si la fecha actual es igual o posterior a la fecha de inicio menos dos días. El límite se ubica exactamente en dos días de anticipación.

Tabla 14. Valores límite de la política de cancelación.

ID	Días de anticipación	Clase	Resultado esperado
CP09	3	Anticipada	Reembolso del 100 % del monto base; las comisiones no se devuelven
CP10	2 (valor límite)	Tardía	Reembolso del 65 %; el 35 % se liquida al anfitrión
CP11	1	Tardía	Reembolso del 65 %; el 35 % se liquida al anfitrión
CP12	0 (día de inicio)	Tardía	Reembolso del 65 %; el 35 % se liquida al anfitrión
CP13	Cualquiera, cancela el anfitrión	Anfitrión	Reembolso del 100 % al cliente

5.6 Diseño de pruebas de caja blanca

Se aplica la técnica del camino básico (McCabe, 1976) sobre ReservaService.cancelar, que incluye la determinación del motivo de cancelación. A partir del grafo de flujo se calcula la complejidad ciclomática y se define un caso de prueba por cada camino independiente.

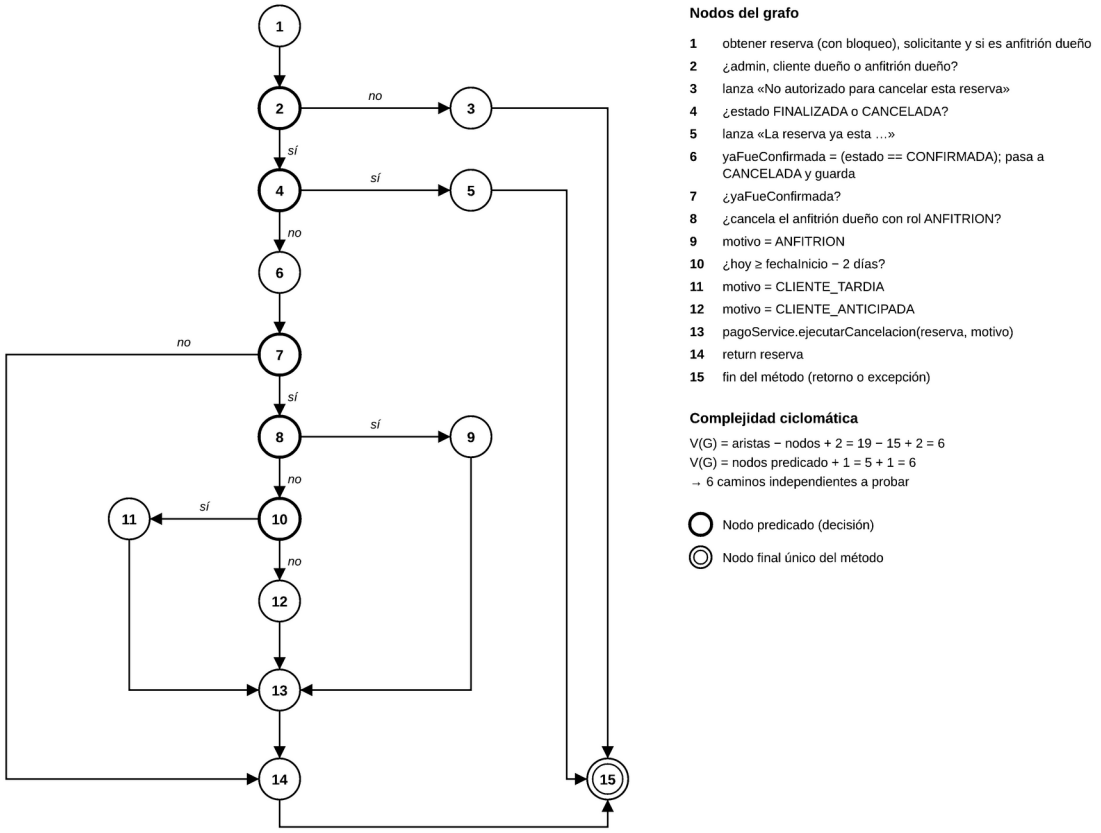


Figura 6. Grafo de flujo de ReservaService.cancelar y su complejidad ciclomática.

Tabla 15. Caminos independientes y casos de prueba de caja blanca.

Camino	**Recorrido (nodos)**	**Datos de entrada**	**Resultado esperado**
C1	1-2-3-15	Usuario que no es admin ni dueño de la reserva	Excepción «No autorizado para cancelar esta reserva.»
C2	1-2-4-5-15	Cliente dueño; reserva FINALIZADA	Excepción «La reserva ya esta FINALIZADA.»
C3	1-2-4-6-7-14-15	Cliente dueño; reserva SOLICITADA	Reserva CANCELADA; no se invoca PagoService (no hay pago que reembolsar)
C4	1-2-4-6-7-8-9-13-14-15	Anfitrión dueño; reserva CONFIRMADA	Reserva CANCELADA; motivo ANFITRION (reembolso total)
C5	1-2-4-6-7-8-10-11-13-14-15	Cliente dueño; CONFIRMADA; inicio = hoy + 1	Reserva CANCELADA; motivo CLIENTE_TARDIA (65 % / 35 %)
C6	1-2-4-6-7-8-10-12-13-14-15	Cliente dueño; CONFIRMADA; inicio = hoy + 5	Reserva CANCELADA; motivo CLIENTE_ANTICIPADA (reembolso total)

La condición compuesta del nodo 2 se completa con dos casos de cobertura de condiciones: un administrador que no es dueño y un anfitrión dueño, ambos autorizados. El camino C3 es el que reproducía el defecto corregido en el commit 57b100d, por lo que este conjunto de pruebas habría detectado ese error.

5.7 Prueba del sistema

Se documenta la prueba de sistema de punta a punta del flujo principal, ejecutada contra el sistema real (backend, base de datos MariaDB, frontend Angular y el entorno de pruebas de MercadoPago).

Tabla 16. Ficha de la prueba de sistema CPS-01.

Campo	Detalle
Identificador	CPS-01 — Ciclo completo de una reserva con pago real
Objetivo	Verificar que una reserva pueda solicitarse, confirmarse, pagarse y reflejar el pago acreditado en la interfaz.
Precondiciones	Servicios en ejecución; un cliente y un anfitrión con cuentas verificadas; credenciales de prueba de MercadoPago; un alojamiento activo publicado.
Fecha de ejecución	03/11/2026

Tabla 17. Pasos y resultados de la prueba de sistema CPS-01.

Paso	Acción	Resultado esperado	Resultado obtenido
1	El cliente inicia sesión y solicita una reserva	Reserva en estado SOLICITADA	Correcto
2	El anfitrión confirma la reserva	Reserva CONFIRMADA; pago PENDIENTE con plazo de 48 h y enlace de pago	Correcto
3	El cliente paga en el checkout con una tarjeta de prueba	MercadoPago aprueba la operación	Correcto
4	Se procesa la notificación de pago	El backend consulta el estado real y marca el pago PAGADO	Correcto (notificación enviada manualmente: en el entorno local la URL de aviso no es accesible desde internet)
5	El cliente abre el detalle de su reserva	Se muestra el pago acreditado y su comprobante	Correcto
6	El anfitrión finaliza la reserva tras la estadía	Reserva FINALIZADA y liquidación al anfitrión	Observación: la API de liquidaciones rechaza la operación con las credenciales de prueba («Invalid signature»); es una limitación de la cuenta, no un defecto del código

Resultado: aprobada con observación. Como antecedente, una ejecución anterior de pruebas sobre el backend detectó un defecto real: cancelar una reserva todavía SOLICITADA fallaba con el mensaje «Esta reserva todavía no tiene un pago asociado». Se corrigió (commit 57b100d) y la re-ejecución fue exitosa.

6. Conclusiones

El backend parte de una base sólida (capas bien definidas, DTOs, bloqueo pesimista, Strategy y DIP ya aplicados), pero concentra la complejidad del dominio en dos clases, ReservaService y PagoService, que además están acopladas entre sí. Las propuestas 1, 2 y 6 atacan ese núcleo y son las de mayor retorno; las restantes reducen acoplamiento con proveedores externos y repetición de reglas.

Referencias

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Larman, C. (2004). Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development (3.ª ed.). Prentice Hall.

Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.

McCabe, T. J. (1976). A complexity measure. IEEE Transactions on Software Engineering, SE-2(4), 308–320.

Myers, G. J., Sandler, C. y Badgett, T. (2011). The art of software testing (3.ª ed.). Wiley.

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), version 2.5.1.